

PART V — TOKENS, CONTEXT, AND MODEL CHOICE

Chapter 25

Why Small Models Struggle with Agentic Loops

“A small model is not a weaker version of a large one doing the same thing more cautiously. It is a different kind of reasoner, and an agentic loop asks it to be something it was never quite trained to be.”

Chapter 25 — Why Small Models Struggle with Agentic Loops

Chapter 24 catalogued three failure modes and traced each to a specific fix already shipped in this project. This chapter asks the question underneath all three: why does `llama-3.1-8b-instant` specifically — the model this project chose for development speed under ADR-005 — produce these failures in the first place, when the identical pipeline running on `llama-3.3-70b-versatile` largely does not?

The honest answer is not a single root cause but five compounding ones, each grounded in a real decision or bug this project's own ledgers recorded: limited working-memory capacity, attention dilution under growing context, instruction-following as a fragile learned behavior rather than a hard rule, no explicit internal state tracking, and a specific failure this project observed directly — the model switching from JSON output into Python code under load.

25.1 Parameter count as working-memory capacity

Parameter count is not merely a speed and cost dial. ADR-005's side-by-side comparison — `llama-3.1-8b-instant` at 128K/8K context versus `llama-3.3-70b-versatile` at 128K/32K, with the larger model rated "much better tool-use and instruction-following" at the identical total window size — is evidence that raw context capacity and effective reasoning capacity are two separate resources. Both models can technically accept a 100,000-token prompt; only one of them reliably reasons well across all of it.

A useful, if informal, analogy: The parameter count behaves less like a bigger notebook and more like a bigger working memory. A larger notebook (more context window) lets you write more down. A bigger working memory (more parameters) lets you hold more of what you wrote down in active consideration at once, weighing it against everything else, while composing the next sentence. The 8B model has plenty of notebook — 128K tokens — but a comparatively small working memory to hold what is written in it under active consideration while generating a response.

Analogy — The two-notebook test

Give two people the identical 50-page case file and ask each to draft a decision. Both can physically hold the pages. Only the one with more working memory can keep page 3's constraint active in mind while writing the conclusion on page 50. Context window is the notebook; parameter count is closer to the working memory reading it.

The distinction matters for a practical reason beyond metaphor: it explains why simply buying more context window — a bigger notebook — does not fix a small model's agentic-loop struggles on its own. Groq's 128K-token window is already far larger than any single query in this project's own dry runs actually used. The bottleneck for this project's own Batch 6, 8, and 10 testing observed was never "the prompt didn't fit" — every one of those runs fit comfortably inside the window. The bottleneck was reasoning quality over what did fit, which is a working-memory constraint, not a notebook-size constraint, and no amount of additional context capacity resolves a working-memory limitation.

25.2 Attention dilution as the conversation grows

Chapter 24.1 established the lost-in-the-middle effect generally. The small-model-specific version of that story is dilution: as more tool calls, tool results, and turns accumulate in a session, each individual token's share of the model's finite attention budget shrinks, and a smaller model has less budget to begin with. The same 2,000 tokens of accumulated history that a 70B model attends to comfortably can meaningfully crowd out an 8B model's attention to the system prompt's rules, simply because there is less total attention capacity to distribute across a growing number of tokens.

This is precisely the mechanism ADR-012 names when describing the failure it fixed: "as the system prompt grew with injected thumbsdown history ... the PROCESS instructions were buried in the middle of

the prompt. The 8B model exhibited recency bias — PROCESS instructions far from the generation point were being ignored." Dilution and recency bias are two names for the same underlying resource constraint, observed directly in this project's own prompt-engineering history.

25.3 Instruction following is learned behavior, not a hard rule engine

It is tempting to think of a system prompt's rules as something closer to code — stated once, obeyed absolutely. They are not. Every instruction in ``_ROLE_AND_RULES`` and ``_PROCESS_INSTRUCTIONS`` is a soft statistical bias on the model's next-token distribution, competing against every other bias the model learned during training and every other signal present in the current context. A rule stated clearly is more likely to be followed, not guaranteed to be followed — and "more likely" is exactly the gap smaller models close less reliably than larger ones.

This reframing explains why ADR-010's phase-restricted tool schema — removing ``check_answer_quality`` from ``tool_schemas`` entirely during the RETRIEVE phase, rather than merely instructing the model not to call it early — was the fix that actually worked, where a purely prompt-based instruction had not been sufficient on its own. A hard architectural constraint (the tool literally is not offered) cannot be statistically overridden the way a soft instruction can. Chapter 17.3's boundary between "what the model is offered" and "what the model is told" is the same distinction, and this is the small-model reason that boundary matters as much as it does.

The general principle worth extracting: for any constraint where violating it would be expensive — a premature answer synthesized from training knowledge rather than retrieved evidence, a fabricated citation, an unauthorized side effect — prefer removing the capability outright over instructing the model not to use it. Reserve soft prompt instructions for constraints where the cost of an occasional violation is genuinely tolerable, such as a stylistic preference in the final answer's phrasing. Confusing the two categories — treating a removable capability as merely a documented rule, or treating a stylistic preference as if it needed architectural enforcement — either leaves a real risk unguarded or spends engineering effort where a prompt sentence would have sufficed.

25.3B Where this leaves prompt engineering

None of this makes prompt wording irrelevant — it narrows what prompt wording is actually good for. A soft instruction still meaningfully shifts the model's next-token distribution in the intended direction; it simply cannot be trusted as the sole safeguard against an expensive failure the way a schema-level constraint can. The practical rule this project's own history supports: use prompt instructions to bias behavior in the common case, and use architectural constraints — a narrowed tool schema, a hard iteration cap, a code-level deduplication check — anywhere the failure mode is expensive enough that "usually obeyed" is not good enough.

25.4 No explicit state tracking

An 8B model has no persistent memory of its own prior actions beyond what is literally re-presented to it as text in the current prompt. "How many times have I already called ``retrieve_documents``" is not a fact the model tracks internally — it is a fact the model must re-derive, every single turn, by re-reading its own accumulated conversation history and counting. Under Section 25.2's dilution, that counting becomes less reliable exactly when history has grown largest — which is precisely when an accurate count matters most.

This project's own architecture does not ask the model to count reliably at all. ``iterations`` is a plain Python integer in ``run_agent()``, incremented once per loop pass and compared against ``MAX_ITERATIONS`` in code — ``while iterations < MAX_ITERATIONS:`` — entirely outside the model's own reasoning. ``seen_queries``, the in-batch deduplication set, is built the same way: a Python ``set()`` populated and checked by the harness, not a fact the model is trusted to track by re-reading its own transcript. Every hard

limit this project enforces on the agentic loop is tracked in code specifically because the alternative — trusting the model's self-reported count — was never a reliable option for an 8B model.

```
iterations = 0
...
while iterations < MAX_ITERATIONS:
    iterations += 1
    ...
    seen_queries: set[str] = set()
    for tc in resp_tool_calls[:MAX_TOOL_CALLS_PER_ITERATION]:
        if q in seen_queries:
            continue # deduplicated by the harness, not the model
```

25.5 The text-vs-code mode-flip failure

This project's own prompt files carry direct evidence of a specific, observed small-model failure mode: several prompts in ``prompts.py`` — ``_DC_SCAN_PROMPT`` and ``_REDUNDANCY_JUDGE_PROMPT`` among them — contain the identical defensive phrasing: "You are NOT writing software. You are NOT generating Python. You are NOT solving a coding task." That phrasing would not exist, worded that specifically and repeated across multiple prompts, unless the model had genuinely produced Python code in place of the requested JSON output under some observed conditions.

The likely mechanism: a small model trained heavily on code alongside natural language can, under context pressure or an ambiguously-phrased instruction, statistically drift toward its code-completion behavior instead of its structured-data-extraction behavior — the two learned patterns are close enough in the model's training distribution that dilution (Section 25.2) can tip the balance. Figure 25.2 traces the failure and the fix side by side.

Both ``_DC_SCAN_PROMPT`` and ``_REDUNDANCY_JUDGE_PROMPT`` are exactly the prompts where the surrounding instructions are already dense with structural, near-code-like language — nested JSON shapes, bracket-counting rules, indexed arrays — which is itself a plausible contributor to why these two prompts specifically needed the reinforced negative instruction rather than every prompt in the file equally.

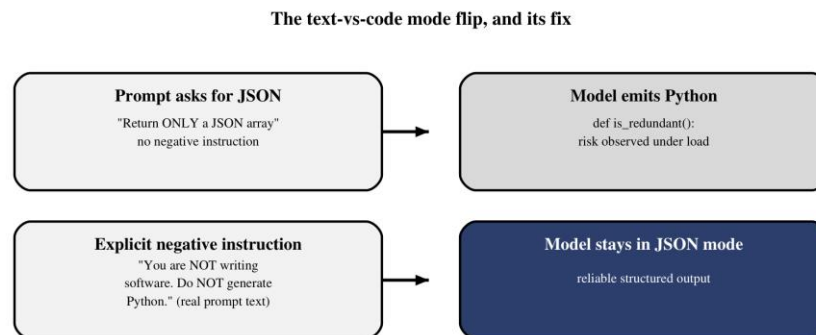


Figure 25.2 — An explicit negative instruction, not a longer positive one, is what closes the text-vs-code mode-flip gap this project's own prompts guard against.

The fix pattern is notable for what it is not: it is not a longer or more elaborate positive instruction ("please return valid JSON matching this schema precisely"), which this project's prompts already stated. It is an explicit negative instruction naming the specific wrong behavior directly. Chapter 26.8 generalizes this observation into a reusable prompt-engineering principle for small models.

25.6 When to upgrade to a 70B model and what changes

Figure 25.1 places the two models' relevant ceilings side by side — not as a recommendation to always prefer the larger model, but as a concrete before/after of what actually changes on upgrade. ADR-018's two-LLM design already anticipated a selective answer: ``judge_llm`` and ``merge_llm``, whose calls demand careful, low-error-tolerance reasoning over compressed context, are the natural first candidates for a 70B swap, while the main orchestration ``llm`` — issuing short, repetitive tool calls — can often stay on the faster, cheaper 8B model without hitting the failure modes this chapter catalogued.

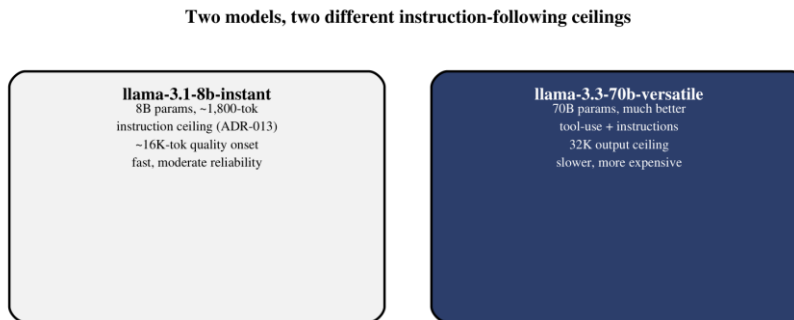


Figure 25.1 — The 70B model's advantage is specifically in instruction-following reliability, not merely a bigger window at the same total context size.

A useful heuristic for deciding which role to upgrade first: ask which role's mistakes are cheapest to recover from. A bad retrieval query costs one wasted call and gets corrected by the next reformulation attempt. A bad judge verdict propagates directly into whether a flawed answer reaches the user. Upgrade the role whose errors are least recoverable first — which is exactly why ``judge_llm``, not the orchestrating ``llm``, is ADR-018's stated first candidate.

It is worth being explicit about the cost side of this decision too, since Chapter 24.7 already raised it: a 70B model is slower and more expensive per call, and a hybrid deployment pays that cost on every judge and merge invocation, not just the ones that would have gone wrong on the 8B model. The upgrade is worth its cost specifically because judge and merge errors are the least recoverable in this pipeline — an unfaithful merge or a wrongly-approved draft has no downstream stage left to catch it — not because a larger model is unconditionally better value for every role in the system.

25.7 Why this isn't a case against small models

None of Sections 25.1 through 25.5 is an argument that small models are unsuitable for agentic RAG. It is an argument that small models need a narrower job description per turn than a monolithic free-form agent loop gives them — exactly the argument ADR-010 already made and this project already shipped. The 4-phase state machine (RETRIEVE → COMPRESS → DRAFT → JUDGE) does not make the 8B model smarter; it makes each individual decision the model is asked to make smaller and more constrained, which is precisely the kind of task a small model's more limited working memory and less reliable instruction-following can still handle well.

This reframes the entire chapter's diagnosis into a design principle: every limitation catalogued here is a reason to narrow what a small model is asked to decide in any single turn, not a reason to abandon it. Chapter 26 takes that principle and applies it specifically to prompt structure — how to phrase, order, and trim what a small model reads so that even within a narrowed phase, its limited attention budget is spent on the instructions that matter most.